Python for the Signals & Systems Lab

A Practical Hand-note: Core Python, NumPy & Matplotlib

Built around the CSE 220 online lab problems (Sections A, B, C)

Topics: input / output • data types • control flow • functions & classes • NumPy arrays & vectorization • Matplotlib plotting • signal operations (time scaling, interpolation, time shift, phase change, time reversal, even/odd decomposition)

Use this as a reference while solving the lab tasks. Every concept comes with a small, runnable example and its expected output.

Contents

1. Running Python & the basic structure of a program
2. Input and Output (print, f-strings, input, formatting numbers)
3. Core data types: int, float, str, bool, list, tuple, dict, set
4. Operators, comparisons and truthiness
5. Control flow: if / elif / else, for, while, break, continue
6. Functions: def, arguments, defaults, return, type hints
7. Comprehensions, enumerate, zip, sorted, min / max with key
8. Classes and objects (attributes, methods, __init__, __str__)
9. NumPy part 1: creating arrays, dtype, shape, indexing, slicing
10. NumPy part 2: vectorization, broadcasting, aggregation, axis
11. NumPy part 3: boolean masks, np.where, clip, searchsorted, sort/argsort
12. Matplotlib part 1: line, scatter, bar, stem, hist
13. Matplotlib part 2: subplots, labels, legend, grid, suptitle, show
14. Signal recipe A: time sub-scaling with interpolation
15. Signal recipe B: time shift vs phase change in a sinusoid
16. Signal recipe C: time reversal and even / odd decomposition
17. Common mistakes & a pre-submission checklist

1. Running Python & the basic structure of a program

A Python program is just a text file ending in .py. You run it from a terminal with `python filename.py`. Code runs top to bottom. Indentation (4 spaces) is not cosmetic in Python — it defines which lines belong to a block.

Most lab templates put the real work inside a main() function and end with the guard below. That guard means “only run main() when this file is executed directly”, which is the standard idiom.

```
import numpy as np
import matplotlib.pyplot as plt


def main():
    print("Hello, signals!")


if __name__ == "__main__":   # runs only when you execute this file
    main()
```

Tip. The two import lines appear in every lab file. `np` and `plt` are just short nicknames so you can write `np.array(...)` instead of `numpy.array(...)`.

2. Input and Output

2.1 Printing

`print(...)` writes to the screen and adds a newline. You can print several things separated by commas; Python inserts a space between them.

```
print("Average:", 46500.5)          # Average: 46500.5
print("a", "b", "c")                 # a b c
print("no newline", end="")          # stays on same line
print(" -> continues here")
```

2.2 f-strings (the clean way to format)

An f-string starts with `f` before the quotes. Anything inside `{ }` is evaluated and inserted. Add `:.2f` to show exactly 2 decimal places — useful for money, averages, MSE, etc.

```
name = "Alice"
avg  = 46500.5
print(f"Employee: {name}, Average: {avg:.2f}")
# Employee: Alice, Average: 46500.50

k = 2
print(f"y(t) = x(t / {k})")          # y(t) = x(t / 2)

mse = 2.5012662848471106e-31
print(f"MSE = {mse:.3e}")            # MSE = 2.501e-31  (scientific)
```

2.3 Reading input

`input()` always returns a string. Convert it with `int(...)` or `float(...)`. To read several numbers on one line, `split()` then map a converter.

```
n = int(input())                       # one integer

# one line: "80 70 90"  ->  [80, 70, 90]
row = list(map(int, input().split()))

# read a name and two ints on one line: "Alice 5 30"
parts = input().split()
name = parts[0]
severity = int(parts[1])
treatment = int(parts[2])
```

Common bug. Forgetting to convert: `input()` gives you `"5"` (text), and `"5" + 1` raises a TypeError. Always wrap numeric input in `int()` / `float()`.

3. Core data types

Python has a handful of built-in types you will use constantly. The table lists what each is for; examples follow.

```
x = 10            # int      whole number
y = 3.14          # float    decimal number
name = "Alice"    # str      text (single or double quotes)
flag = True       # bool     True / False

nums  = [1, 2, 3]         # list   ordered, CHANGEABLE
point = (35.7, 139.7)     # tuple  ordered, FIXED (cannot change)
ages  = {"Alice": 30}     # dict   key -> value lookup
seen  = {1, 2, 3}         # set    unordered, no duplicates
```

3.1 Lists (your everyday container)

```
salaries = []                 # empty list
salaries.append(45000)        # add to the end
salaries.append(46000)
print(salaries)               # [45000, 46000]
print(len(salaries))          # 2   (how many items)
print(salaries[0])            # 45000  (first, index starts at 0)
print(salaries[-1])           # 46000  (last)
print(sum(salaries))          # 91000
print(max(salaries))          # 46000
```

3.2 Tuples (fixed records)

A tuple groups related values that belong together and should not change — for example one patient visit as `(name, severity, time)`. You can 'unpack' a tuple straight into variables.

```
visit = ("Alice", 5, 30)
name, severity, treatment = visit    # unpacking
print(name)        # Alice
print(severity)    # 5

# a list of tuples is a very common pattern
visits = [("Alice", 5, 30), ("Bob", 3, 20)]
for name, sev, t in visits:
    print(name, sev, t)
```

3.3 Dictionaries (key -> value)

```
summary = {}
summary["Alice"] = {"total_time": 0, "total_sev": 0}
summary["Alice"]["total_time"] += 30      # update in place

# setdefault: get the entry, creating it if missing (one-liner)
entry = summary.setdefault("Bob", {"total_time": 0, "total_sev": 0})
entry["total_time"] += 20

for name in sorted(summary):              # keys in alphabetical order
    print(name, summary[name])
```

4. Operators, comparisons and truthiness

```
# arithmetic
7 + 2      # 9
7 - 2      # 5
7 * 2      # 14
7 / 2      # 3.5     (always float)
7 // 2     # 3       (floor / integer division)
7 % 2      # 1       (remainder / modulo)
2 ** 10    # 1024    (power)

# comparison -> gives a bool
5 > 3      # True
5 == 5     # True    (== is equality;  = is assignment)
5 != 4     # True

# logic (for single True/False values)
(5 > 3) and (2 < 1)    # False
(5 > 3) or  (2 < 1)    # True
not True               # False
```

Watch out. For plain Python booleans use `and` / `or` / `not`. But for NumPy arrays you must use `&`, `|`, `~` instead (covered in the NumPy section). Mixing them up is one of the most common NumPy errors.

5. Control flow

5.1 if / elif / else

Used, for example, to convert a percentage into a grade — exactly the Section-style task.

```
def grade(pct):
    if pct >= 90:
        return "Excellent"
    elif pct >= 75:
        return "Good"
    elif pct >= 60:
        return "Average"
    else:
        return "Poor"

print(grade(92))   # Excellent
print(grade(64))   # Average
```

5.2 for loops

```
for i in range(5):          # 0, 1, 2, 3, 4
    print(i, end=" ")
print()                     # 0 1 2 3 4

months = ["Jan", "Feb", "Mar"]
for m in months:            # loop over items directly
    print(m)

for i, m in enumerate(months):   # index AND item together
    print(i, m)             # 0 Jan / 1 Feb / 2 Mar
```

5.3 while loops

A `while` loop repeats as long as its condition is true. The time-reversal task swaps items from both ends moving inward.

```
i = 0
mid = (n - 1) / 2.0
while i <= mid:
    xr[i], xr[n-1-i] = x[n-1-i], x[i]   # swap two ends
    i += 1
```

6. Functions

A function packages reusable logic. Define it with `def`, give it inputs (parameters), and hand back a result with `return`. The type hints (e.g. `n: int`, `-> np.ndarray`) are optional labels that document what goes in and comes out; they do not change behaviour but the templates use them.

```
def sinusoid(n: np.ndarray, A: float, Omega0: float, phi: float) -> np.ndarray:
    """A cos(Omega0 * n + phi)  -- docstring explains the function."""
    return A * np.cos(n * Omega0 + phi)

# call it
import numpy as np
n = np.arange(-3, 4)
x = sinusoid(n, A=1.0, Omega0=np.pi/4, phi=0.0)
print(x.round(3))
```

Arguments can be passed by position or by name. Default values make a parameter optional.

```
def power(base, exp=2):     # exp defaults to 2
    return base ** exp

print(power(5))            # 25   (uses default exp=2)
print(power(5, 3))         # 125  (position)
print(power(base=5, exp=3))# 125  (by name)
```

7. Comprehensions, enumerate, zip, sorted, min / max

7.1 List comprehension

A compact way to build a list by transforming or filtering another sequence.

```
squares = [x*x for x in range(5)]          # [0, 1, 4, 9, 16]
evens   = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]
grades  = [grade(p) for p in [92, 80, 55]] # ['Excellent','Good','Poor']
```

7.2 sorted, and min / max with a key

`sorted(...)` returns a new sorted list. The key argument lets you sort (or pick min/max) by a computed value. A neat trick: to get the highest score but break ties by smallest name, sort by the tuple `(-score, name)`.

```
scores = {"Alice": 12, "Bob": 9, "Charlie": 9}

# alphabetical
for name in sorted(scores):
    print(name, scores[name])

# highest score; tie -> lexicographically smaller name
top = min(scores, key=lambda nm: (-scores[nm], nm))
print("TOP", top, scores[top])       # TOP Alice 12
```

7.3 zip

```
names = ["Alice", "Bob"]
ages  = [30, 25]
for name, age in zip(names, ages):
    print(name, age)     # Alice 30 / Bob 25
```

8. Classes and objects

A class is a blueprint for objects that bundle data (attributes) with behaviour (methods). `__init__` runs when you create an object and sets up its attributes. `self` refers to the particular object. `__str__` controls what `print(obj)` shows.

```
class Employee:
    company_name = "TechCorp"          # CLASS attribute (shared by all)

    def __init__(self, name, emp_id):  # runs at creation time
        self.name = name               # INSTANCE attributes (per object)
        self.emp_id = emp_id
        self.salaries = []             # own list for each employee

    def add_salary(self, salary):
        self.salaries.append(salary)

    def average_salary(self):
        return sum(self.salaries) / len(self.salaries)

    def __str__(self):
        return f"{self.name} ({self.emp_id}): avg = {self.average_salary():.2f}"


e = Employee("Alice", "E101")
e.add_salary(45000)
e.add_salary(48000)
print(e)                    # Alice (E101): avg = 46500.00
```

Why put `salaries` in `__init__` and not as a class attribute? A mutable class attribute would be shared by every employee, so everyone would accidentally see the same salary list. Instance attributes give each object its own independent copy.

9. NumPy part 1: arrays, dtype, shape, indexing

A NumPy ndarray is a grid of values of the same type, stored compactly in one block of memory. That is what makes whole-array math fast. Compared with a Python list: a list holds pointers to scattered objects of any type; an ndarray holds raw numbers back-to-back and supports element-wise math directly.

```
import numpy as np

a = np.array([1, 2, 3])            # 1-D array
b = np.array([[1, 2], [3, 4]])     # 2-D array (matrix)

print(a.shape)     # (3,)      -> 3 elements
print(b.shape)     # (2, 2)    -> 2 rows, 2 cols
print(a.dtype)     # int64     -> the element type

# handy constructors
np.zeros(4)                 # [0. 0. 0. 0.]
np.ones((2, 3))             # 2x3 of ones
np.arange(-3, 4)            # [-3 -2 -1  0  1  2  3]
np.linspace(-4, 4, 9)       # 9 evenly spaced points from -4 to 4
```

Indexing and slicing look like lists, but 2-D arrays use a single bracket with a comma.

```
a = np.array([10, 20, 30, 40, 50])
a[0]        # 10
a[-1]       # 50
a[1:4]      # [20 30 40]   (start inclusive, stop exclusive)

b = np.array([[1, 2, 3], [4, 5, 6]])
b[1, 2]     # 6            (row 1, col 2)
b[:, 1]     # [2 5]        (all rows, column 1)
b[0, :]     # [1 2 3]      (row 0, all columns)
```

10. NumPy part 2: vectorization, broadcasting, axis

Vectorization means applying an operation to a whole array at once, with no Python loop. It is shorter and much faster.

```
a = np.array([1, 2, 3, 4])
a * 2          # [2 4 6 8]      every element doubled
a + 10         # [11 12 13 14]
a ** 2         # [1 4 9 16]
np.sin(a)      # sine of each element

# CAREFUL: on a Python list, * repeats instead of multiplying!
[1, 2, 3] * 2  # [1, 2, 3, 1, 2, 3]   (NOT element-wise)
```

Broadcasting lets arrays of different shapes combine. A common case: divide every column of a matrix by a per-column target vector.

```
A = np.array([[80, 70, 90],
              [60, 85, 75]], dtype=float)
target = np.array([100, 100, 100], dtype=float)

P = 100 * A / target      # each column divided by its target
print(P)
```

The axis argument chooses the direction of an aggregation. `axis=0` collapses rows (result per column); `axis=1` collapses columns (result per row).

```
P.mean(axis=1)      # average per ROW    (per salesperson)
P.mean(axis=0)      # average per COLUMN (per product)
P.sum()             # grand total (no axis -> whole array)
P.argmax(axis=1)    # index of the max in each row
np.argsort(-P.mean(axis=1))   # indices sorted high -> low
```

Reshape trick. To turn 12 monthly values into 4 quarterly sums: `monthly.reshape(4, 3).sum(axis=1)` groups them into 4 rows of 3 and sums each row.

11. NumPy part 3: masks, where, clip, searchsorted

11.1 Boolean masks

```
t = np.array([-2, -1, 0, 1, 2])
mask = np.abs(t) <= 1           # [F  T  T  T  F]
t[mask]                         # [-1  0  1]   select where True
t[mask] = 0                     # assign only where True

# combine conditions with &  and  | (NOT and/or), with parentheses
both = (t >= 0) & (t <= 1)
```

11.2 np.where (vectorized if/else)

`np.where(cond, a, b)` picks from `a` where the condition is true, otherwise from `b` — element by element.

```
x = np.array([-3, -1, 2, 5])
np.where(x >= 0, x, 0)      # [0 0 2 5]   clamp negatives to 0
np.where(x >= 0, 1, -1)     # [-1 -1 1 1] sign as +/-1
```

11.3 clip, sort, argsort

```
np.clip([-2, 5, 12], 0, 9)    # [0 5 9]   force into [0, 9]

v = np.array([30, 10, 20])
np.sort(v)                    # [10 20 30]   sorted values
np.argsort(v)                 # [1 2 0]      indices that sort v
```

11.4 searchsorted (binary search for nearest neighbours)

Given a sorted array, `np.searchsorted` finds, for each query, the position where it would be inserted to keep order. The element just before that position is the nearest smaller value; the element at it is the nearest larger. This is the heart of the interpolation task.

```
b = np.array([10, 20, 30, 40, 50])
q = np.array([15, 42, 7])
idx = np.searchsorted(b, q)   # [1 2 0]

n = len(b)
left  = b[np.clip(idx - 1, 0, n-1)]   # nearest smaller
right = b[np.clip(idx,     0, n-1)]   # nearest larger
# average of neighbours = linear-ish interpolation
mid = 0.5 * (left + right)
```

12. Matplotlib part 1: the plot types

All examples assume `import matplotlib.pyplot as plt`. Nothing appears on screen until you call `plt.show()` at the end.

12.1 Line plot

```
plt.plot(x, y, linestyle="-",  color="blue",  label="North")
plt.plot(x, z, linestyle="--", color="green", label="South")
plt.plot(x, w, linestyle=":",  color="red",   label="Central")
# linestyle: "-" solid, "--" dashed, ":" dotted, "-." dash-dot
# marker (separate!): "o" circle, "s" square, "^" triangle
```

Classic mix-up. `"-"`, `"--"`, `":"` are linestyles, not markers. Passing them to `marker=` will not draw the dashed line you expect.

12.2 Scatter

```
plt.scatter(x, y, color="blue", s=60)   # s = marker SIZE (not "size")
```

12.3 Bar (and colour pitfall)

```
branches = ["North", "South", "Central"]
values   = [195, 180, 175]
plt.bar(branches, values, color=["blue", "green", "red"])
# NOTE: the 3rd POSITIONAL arg of bar() is WIDTH, so pass color=... by name
```

12.4 Stacked bar (stack segments with bottom=)

```
plt.bar(q, north, color="blue",  label="North")
plt.bar(q, south, bottom=north,        color="green", label="South")
plt.bar(q, central, bottom=north+south, color="red",  label="Central")
# each layer starts where the ones below it end (use numpy arrays so + adds)
```

12.5 Histogram

```
plt.hist([north, south, central], bins=8,
         color=["blue", "green", "red"],
         label=["North", "South", "Central"])
```

13. Matplotlib part 2: subplots, labels, legend, show

Every plot should have a title, axis labels, a legend (when there are multiple series) and a grid. For several plots in one figure, use `plt.subplots` to get a grid of axes.

```
fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(14, 10))

ax[0, 0].plot(x, y, label="signal")
ax[0, 0].set_title("Top-left plot")
ax[0, 0].set_xlabel("t")
ax[0, 0].set_ylabel("Amplitude")
ax[0, 0].legend()
ax[0, 0].grid(True)          # grid(axis="y") for horizontal lines only

fig.suptitle("Overall Figure Title", fontsize=16)   # one big title
plt.tight_layout()           # stop labels/titles overlapping
plt.show()                   # <-- REQUIRED, or nothing appears
```

Stem plots (for discrete signals) return three pieces: `markerline, stemlines, baseline = ax.stem(n, x, label=...)`. Hide the baseline with `baseline.set_visible(False)` for a cleaner look.

If `fig.suptitle` overlaps the top row, reserve space with `plt.tight_layout(rect=[0, 0, 1, 0.96])`.

14. Signal recipe A: time sub-scaling with interpolation

Goal. Produce y(t) = x(t / k) from sampled data. Some required points fall between known samples (e.g. y(1) needs x(0.5)). We fill those in as the average of the nearest left and right known samples.

Idea. For each query time, use `searchsorted` to locate the surrounding known samples, then average their x-values. Clip indices so the array edges stay in range, and mark anything outside the known range as NaN (Matplotlib skips NaN).

```
def interpolate_signal(t_original, x_original, t_query):
    order = np.argsort(t_original)          # ensure sorted
    ts, xs = t_original[order], x_original[order]
    n = len(ts)

    idx = np.searchsorted(ts, t_query, side="left")
    idc = np.clip(idx, 0, n - 1)

    exact = (idx < n) & np.isclose(ts[idc], t_query)   # lands on a sample?
    left  = xs[np.clip(idx - 1, 0, n - 1)]
    right = xs[np.clip(idx,     0, n - 1)]
    avg   = 0.5 * (left + right)

    result = np.where(exact, xs[idc], avg)
    outside = (t_query < ts[0]) | (t_query > ts[-1])
    return np.where(outside, np.nan, result)


def time_scale(t, x, k):
    return interpolate_signal(t, x, t / k)   # query x at t/k
```

Why average the neighbours? The task defines the missing value as y(1) = 0.5 × (x(0) + x(1)). That is exactly the midpoint of the two nearest known samples, which is what the code computes for every non-exact query at once.

15. Signal recipe B: time shift vs phase change

Signal. $x[n] = A \cos(\Omega_0 n + \varphi)$. A time shift replaces n by $n + n_0$; a phase change adds φ_0 to the angle.

```
def sinusoid(n, A, Omega0, phi):
    return A * np.cos(n * Omega0 + phi)

def time_shift_sinusoid(n, A, Omega0, phi, n0):
    return sinusoid(n + n0, A, Omega0, phi)          # x[n + n0]

def phase_change_sinusoid(n, A, Omega0, phi, phi0):
    return sinusoid(n, A, Omega0, phi + phi0)        # add phi0
```

Key relationship. Shifting by n_0 is identical to adding phase $\varphi_0 = \Omega_0 \times n_0$, because $\cos(\Omega_0(n + n_0) + \varphi) = \cos(\Omega_0 n + \varphi + \Omega_0 n_0)$. So every integer time shift has an exact phase equivalent (their MSE is ~0).

The reverse is not always true. An arbitrary phase change corresponds to a time shift $n_0 = \varphi_0 / \Omega_0$, which is usually not an integer. Since n must be an integer in discrete time, you can only find the closest integer shift, so the MSE is small but non-zero. Also, shifts repeat with the signal's period, so several integer shifts can tie for best.

```
# demonstrate: search integer shifts for the best match to a phase change
best_k, best_err = None, None
for k in range(-12, 13):
    e = mse(time_shift_sinusoid(n, A, Omega0, phi, k), x_phase)
    if best_err is None or e < best_err:
        best_err, best_k = e, k
print(best_k, best_err)
```

16. Signal recipe C: time reversal & even / odd decomposition

Time reversal gives x(−t): the sample at +t moves to −t. On a symmetric sample grid this is just flipping the array. Even / odd decomposition then splits any signal into a symmetric part and an anti-symmetric part.

```
def time_reverse(x):
    xr = x.copy()               # COPY first -- do not mutate the input!
    n = len(x)
    i = 0
    mid = (n - 1) / 2.0
    while i <= mid:
        xr[i], xr[n-1-i] = x[n-1-i], x[i]   # swap ends inward
        i += 1
    return xr
    # (equivalently: return x[::-1].copy())

def even_odd_decompose(x):
    xr = time_reverse(x)
    xe = 0.5 * (x + xr)         # even part  = symmetric
    xo = 0.5 * (x - xr)         # odd part   = anti-symmetric
    return xe, xo
```

The single most important bug to avoid here: writing `xr = x` instead of `xr = x.copy()`. Plain assignment makes `xr` another name for the same array, so swapping in place also scrambles the original `x`. That makes the odd part come out as all zeros. Always copy before an in-place edit.

Sanity checks (great to show the instructor): the even part satisfies xe(t) = xe(−t), the odd part satisfies xo(t) = −xo(−t), and xe + xo reconstructs the original x exactly.

```
print(np.max(np.abs(xe - time_reverse(xe))))   # ~0  (even)
print(np.max(np.abs(xo + time_reverse(xo))))   # ~0  (odd)
print(np.max(np.abs(x - (xe + xo))))           # ~0  (reconstruct)
```

17. Common mistakes & a pre-submission checklist

- Forgot `plt.show()`. The figures are built but never displayed. Always end with it.
- `marker="-"`. Dashes/dots are linestyles; use `linestyle=`. Markers are letters like `"o"`.
- `scatter(size=...)`. The size keyword is `s`, not `size`.
- `bar(x, h, color_list)`. The 3rd positional arg is width. Pass `color=` by name.
- List `+` instead of array `+`. `[1,2] + [3,4]` concatenates; convert to `np.array` for element-wise addition (matters for stacked-bar `bottom=`).
- `xr = x` aliasing. Use `x.copy()` before editing in place, or the original is destroyed.
- Using `and/or` on arrays. Use `&` / `|` with parentheses for NumPy boolean masks.
- Index out of range from `searchsorted`. It can return `len(array)`; `np.clip` the indices before using them.
- Forgot to convert `input()`. It returns text; wrap in `int()` / `float()`.
- Missing title / labels / legend / grid. These usually carry marks in the plotting rubric.

Before you submit: run the file top to bottom with no errors; check every subplot has a title, x-label, y-label, legend and grid; confirm the figure shows on screen; and read the task sheet once more to match the exact wording (titles, ranges, k value).